

RATL Specification

RATL (R-based Array Manipulation Language) is a stack-based, esoteric programming language inspired by [MATL](#). It leverages R's powerful statistical and matrix capabilities through a concise, postfix syntax, making it highly suitable for code golf.

Total symbols: 420 (all 2-byte or shorter)

1. Introduction

RATL is implemented in R and uses R's internal data structures (vectors, lists, matrices) to handle stack operations. It translates RATL code into R code, which is then evaluated.

1.1 The Name

The name stands for **R-based Array Manipulation Language**, reflecting its foundation in R and its primary focus on array-based operations.

1.2 Notation

- **\$n**: Represents the n -th element popped from the stack (starting from the top).
 - **stack**: The global list used for storage.
 - **H, G, L, M**: Clipboards for temporary storage outside the stack.
-

2. The Stack and Data Types

RATL is a stack-based language where every operation interacts with a global stack (an R `list`).

Data Types

RATL uses native R types: - **Numeric**: Integers and floating-point numbers (R `numeric`). - **Logical**: Boolean values (`TRUE`, `FALSE`). - **Character**: Strings of text (R `character`). - **List/Cell Array**: Collections of items, which can be of mixed types (R `list`). - **Matrix**: 2D arrays (R `matrix`).

3. Statements and Separators

- **Space:** Acts as a token separator.
 - **Newline:** Ignored by the parser but acts as a token separator.
 - **Comments:** Lines starting with `#` are treated as comments and ignored.
-

4. Literals

Literals are pushed onto the stack as soon as they are encountered.

4.1 Numbers

Standard numeric notation: 1, 10, 3.14, -5.

4.2 Numerical Arrays (Vectors)

Enclosed in square brackets `[...]`. Elements are separated by spaces. Example: `[1 2 3]` pushes a numeric vector of length 3.

4.3 Character Arrays (Strings)

Enclosed in single quotes `'...'`. Example: `'hello'` pushes a character vector of length 1.

4.4 Cell Arrays (Lists)

Enclosed in curly braces `{...}`. Example: `{1 'a' [1 2]}` pushes an R list containing three different types.

5. Control Flow

RATL uses special symbols for conditional branching and loops.

5.1 Conditional Branching (? ...])

- **Symbols:** ? starts the block,] ends it.
- **If-Else:** ? `truthy` ; `falsy`]. The ; separator distinguishes the “then” and “else” branches.
- **Logic:** Pops the top element. If it’s true, it executes the code until] (or ;). If false and ; is present, it executes the code between ; and].

5.2 While/Repeat Loop (" ... ")

- **Symbol:** "
- **Logic:** Starts a loop. At the end of the loop (marked by another "), it pops the top element. If it’s true, it repeats.

5.3 For-Each Loop ((...))

- **Symbols:** (and)
- **Logic:** (pops an array/list. It then iterates through each element, pushing it onto the stack and executing the code between (and).

5.4 Infinite Loop (` ... `)

- **Symbols:** ` starts and ends the loop.
- **Logic:** Executes the code within the backticks repeatedly. Use the **X** (break) symbol to exit.

5.5 Higher-Order Functions (Block Consumers)

These functions pop a {block} from the stack and apply it to data.

Symbol	Name	Description
q	Map	{block} array q — applies block to each element
e	Filter	{block} array e — keeps elements where block returns <code>truthy</code>
y	Reduce	{block} array y — fold left with block
z	Repeat	N {block} z — executes block N times
@	Execute	{block} @ — pops and executes a block immediately

6. Implicit Actions

6.1 Initial Actions

- The stack is initialized as an empty `list()`.
- A connection to `stdin` is opened for input operations.

6.2 Final Actions

- **Implicit Print:** If the stack contains exactly one element, it is printed automatically. If it contains more than one, the entire stack is printed as a list.
 - **Invisible Output:** The final evaluation result is returned invisibly in R.
-

7. Symbol Reference

7.1 Arithmetic & Comparison

Symbol	R Code	Description	Stack Effect
%	<code>\$2 %% \$1</code>	Modulo	$2 \rightarrow 1$
*	<code>\$2 * \$1</code>	Multiply	$2 \rightarrow 1$
+	<code>\$2 + \$1</code>	Add	$2 \rightarrow 1$
-	<code>\$2 - \$1</code>	Subtract	$2 \rightarrow 1$
/	<code>\$2 / \$1</code>	Divide	$2 \rightarrow 1$
<	<code>\$2 < \$1</code>	Less	$2 \rightarrow 1$
<=	<code>\$2 <= \$1</code>	LessEqual	$2 \rightarrow 1$
=	<code>\$2 == \$1</code>	Equal	$2 \rightarrow 1$
>	<code>\$2 > \$1</code>	Greater	$2 \rightarrow 1$
>=	<code>\$2 >= \$1</code>	GreatEqual	$2 \rightarrow 1$
^	<code>\$2 ^ \$1</code>	Power	$2 \rightarrow 1$
lX	<code>xor(\$2, \$1)</code>	Xor	$2 \rightarrow 1$
mI	<code>\$2 %/% \$1</code>	IntDiv	$2 \rightarrow 1$
\	<code>\$2 \ \$1</code>	Or	$2 \rightarrow 1$
~	<code>!\$1</code>	Not	$1 \rightarrow 1$

7.2 Stack & Control

Symbol	R Code	Description	Stack Effect
D	<code>list(\$1, \$1)</code>	Duplicate	$1 \rightarrow 2$
Ls	<code>length(stack)</code>	Stack Length	$0 \rightarrow 1$
U	<code>for(x in \$1) stack[[length(stack)+...</code>	Unpack	$1 \rightarrow 0$
i	<code>scan(ratl_stdin, what=character(),...</code>	Input	$0 \rightarrow 1$
p	<code>ratl_print(\$1)</code>	Print	$1 \rightarrow 0$
sC	<code>cat(\$1)</code>	Cat	$1 \rightarrow 0$
sM	<code>message(\$1)</code>	Message	$1 \rightarrow 0$
sS	<code>stop(\$1)</code>	Stop	$1 \rightarrow 0$
sW	<code>warning(\$1)</code>	Warning	$1 \rightarrow 0$
w	<code>list(\$1, \$2)</code>	Swap	$2 \rightarrow 2$
x	NULL	Delete	$1 \rightarrow 0$

7.3 Constants

Symbol	R Code	Description	Stack Effect
In	<code>Inf</code>	Inf	$0 \rightarrow 1$
Na	<code>NA</code>	NA	$0 \rightarrow 1$
Pi	<code>pi</code>	Pi	$0 \rightarrow 1$
s4	<code>logical(0)</code>	EmptyLog	$0 \rightarrow 1$
sQ	<code>character(0)</code>	EmptyChar	$0 \rightarrow 1$
sZ	<code>numeric(0)</code>	EmptyNum	$0 \rightarrow 1$

7.4 Higher-Order Functions

Symbol	R Code	Description	Stack Effect
Fa	<code>apply(\$3, \$2, \$1)</code>	Apply	$3 \rightarrow 1$
Ff	<code>Filter(\$2, \$1)</code>	Filter (Func)	$2 \rightarrow 1$
F1	<code>lapply(\$2, \$1)</code>	Lapply	$2 \rightarrow 1$
Fm	<code>mapply(\$1, ...)</code>	Mapply	$1 \rightarrow 1$
Fn	<code>Find(\$2, \$1)</code>	Find (Func)	$2 \rightarrow 1$
Fp	<code>Position(\$2, \$1)</code>	Position	$2 \rightarrow 1$
Fq	NULL	Map (evaluator)	$0 \rightarrow 1$
Fr	<code>Reduce(\$2, \$1)</code>	Reduce	$2 \rightarrow 1$
Fs	<code>sapply(\$2, \$1)</code>	Sapply	$2 \rightarrow 1$
Ft	NULL	Filter (evaluator)	$0 \rightarrow 1$

Symbol	R Code	Description	Stack Effect
Fv	vapply(\$3, \$2, \$1)	Vapply	$3 \rightarrow 1$
Fx	NULL	Repeat (evaluator)	$0 \rightarrow 1$
e	NULL	Filter (evaluator)	$0 \rightarrow 1$
fn	Negate(\$1)	Negate	$1 \rightarrow 1$
q	NULL	Map (evaluator)	$0 \rightarrow 1$
y	NULL	Reduce (evaluator)	$0 \rightarrow 1$
z	NULL	Repeat (evaluator)	$0 \rightarrow 1$

7.5 Array Operations

Symbol	R Code	Description	Stack Effect
#	\$2[\$1]	Filter	$2 \rightarrow 1$
,	c(\$2, \$1)	Concat Vectors	$2 \rightarrow 1$
.	1:\$1	Range 1 to N	$1 \rightarrow 1$
..	list(\$2, \$1)	Pair	$2 \rightarrow 1$
:	seq(\$2, \$1)	Sequence	$2 \rightarrow 1$
O	sort(\$1)	Sort	$1 \rightarrow 1$
R	rev(\$1)	Reverse	$1 \rightarrow 1$
SH	head(\$1)	Head	$1 \rightarrow 1$
SR	rank(\$1)	Rank	$1 \rightarrow 1$
ST	tail(\$1)	Tail	$1 \rightarrow 1$
XC	cumsum(\$1)	CumSum	$1 \rightarrow 1$
Xk	{r <- rle(\$1); list(r\$values, r\$le...}	RLE	$1 \rightarrow 2$
c1	cumprod(\$1)	CumProd	$1 \rightarrow 1$
cn	is.element(\$2, \$1)	IsIn	$2 \rightarrow 1$
dO	order(\$1)	Order	$1 \rightarrow 1$
dS	split(\$2, \$1)	Split	$2 \rightarrow 1$
dU	unsplit(\$2, \$1)	Unsplit	$2 \rightarrow 1$
e1	\$2[[\$1]]	Extract Element [[	$2 \rightarrow 1$
en	\$2[[as.character(\$1)]]	Extract Name \$	$2 \rightarrow 1$
es	\$2[\$1]	Extract Subset [	$2 \rightarrow 1$
fE	head(\$1, 1)	FirstElem	$1 \rightarrow 1$
fu	unlist(\$1)	Flatten	$1 \rightarrow 1$
hd	head(\$2, \$1)	HeadN	$2 \rightarrow 1$
ix	match(\$2, \$1)	IndexOf	$2 \rightarrow 1$
l	length(\$1)	Length	$1 \rightarrow 1$
la	tail(\$1, 1)	LastElem	$1 \rightarrow 1$
ns	\$2[length(\$2)+1-\$1]	NegSlice	$2 \rightarrow 1$
r1	1:\$1	Range1toN	$1 \rightarrow 1$

Symbol	R Code	Description	Stack Effect
r2	rep(\$2, \$1)	Rep	2 → 1
tb	tabulate(\$1)	Tabulate	1 → 1
tl	tail(\$2, \$1)	TailN	2 → 1
u	unique(\$1)	Unique	1 → 1
un	length(unique(\$1))	UniqueN	1 → 1
vB	pmin(\$2, \$1)	PMin	2 → 1
vD	drop(\$1)	Drop	1 → 1
vF	diff(\$1)	Diff	1 → 1
vG	pmax(\$2, \$1)	PMax	2 → 1
vH	which(\$1, arr.ind=TRUE)	WhichArr	1 → 1
vI	cummin(\$1)	CumMin	1 → 1
vT	matrix(rep(\$3, \$2*\$1), nrow=nrow(\$...	RepMat	3 → 1
vW	seq(\$3, \$2, length.out=\$1)	SeqLen	3 → 1
vX	cummax(\$1)	CumMax	1 → 1
wh	which(\$1)	Which	1 → 1
zp	unlist(Map(list, \$2, \$1))	Zip	2 → 1

7.6 Matrix

Symbol	R Code	Description	Stack Effect
!	t(\$1)	Transpose	1 → 1
&	\$2 %o% \$1	Outer Product	2 → 1
BT	{idx<-c(1,4,3,2,5); \$1[idx,idx]}	Bingo Twin	1 → 1
R9	apply(t(\$1), 2, rev)	Rot90	1 → 1
Y!	matrix(\$3, nrow=\$2, ncol=\$1)	Create Matrix	3 → 1
Y*	\$2 %*% \$1	Matrix Mult	2 → 1
YM	matrix(\$3, nrow=\$2, ncol=\$1, byrow=...	Matrix ByRow	3 → 1
v2	solve(\$2, \$1)	Solve 2	2 → 1
vC	col(\$1)	Col Indices	1 → 1
vE	rowSums(\$1)	Row Sums	1 → 1
vL	solve(\$1)	Solve	1 → 1
vM	colMeans(\$1)	Col Means	1 → 1
vN	rowMeans(\$1)	Row Means	1 → 1
vQ	qr(\$1)	QR Decomp	1 → 1
vR	row(\$1)	Row Indices	1 → 1
vS	colSums(\$1)	Col Sums	1 → 1
vV	svd(\$1)	SVD	1 → 1
y!	diag(\$1)	Diag	1 → 1
yD	diag(\$1)	Identity Matrix	1 → 1

Symbol	R Code	Description	Stack Effect
yc	<code>eigen(\$1)\$vectors</code>	EigenVectors	1 → 1
yd	<code>det(\$1)</code>	Determinant	1 → 1
yf	<code>as.matrix(\$1)</code>	Full	1 → 1
yk	<code>kronecker(\$2, \$1)</code>	Kronecker	2 → 1
yl	<code>{x<-\$1; x[upper.tri(x)]<-0; x}</code>	TriLower	1 → 1
ym	<code>matrix(\$3, nrow=\$2, ncol=\$1)</code>	Create Matrix	3 → 1
yt	<code>sum(diag(\$1))</code>	Trace	1 → 1
yu	<code>{x<-\$1; x[lower.tri(x)]<-0; x}</code>	TriUpper	1 → 1
yv	<code>eigen(\$1)\$values</code>	EigenValues	1 → 1

7.7 Statistics

Symbol	R Code	Description	Stack Effect
B5	<code>fivenum(\$1)</code>	FiveNum	1 → 1
BC	<code>cov(\$2, \$1)</code>	Covariance	2 → 1
BS	<code>summary(\$1)</code>	Summary	1 → 1
Ba	<code>mad(\$1)</code>	MAD	1 → 1
Bc	<code>cor(\$2, \$1)</code>	Correlation	2 → 1
Bi	<code>IQR(\$1)</code>	IQR	1 → 1
Bn	<code>min(\$1)</code>	Min	1 → 1
Br	<code>range(\$1)</code>	Range	1 → 1
Bs	<code>sd(\$1)</code>	Standard Deviation	1 → 1
Bx	<code>max(\$1)</code>	Max	1 → 1
Bz	<code>scale(\$1)</code>	Scale	1 → 1
Ku	<code>sum((\$1-mean(\$1))^4)/((length(\$1)-...</code>	Kurtosis	1 → 1
P	<code>prod(\$1)</code>	Product	1 → 1
Q	<code>quantile(\$1)</code>	Quantile	1 → 1
Sd	<code>ratl_cum_sd(\$1)</code>	CumSD	1 → 1
Sk	<code>sum((\$1-mean(\$1))^3)/((length(\$1)-...</code>	Skewness	1 → 1
Sm	<code>mean(\$1, na.rm=TRUE)</code>	MeanNA	1 → 1
Sn	<code>sum(\$1, na.rm=TRUE)</code>	SumNA	1 → 1
Sw	<code>weighted.mean(\$2, \$1)</code>	WeightedMean	2 → 1
V	<code>var(\$1)</code>	Variance	1 → 1
Xo	<code>ratl_mode(\$1)</code>	Mode	1 → 1
Xr	<code>ratl_rms(\$1)</code>	RMS	1 → 1
Xs	<code>sd(\$1)</code>	Std Dev	1 → 1
cr	<code>cor(\$2, \$1)</code>	Correlation	2 → 1
cv	<code>cov(\$2, \$1)</code>	Covariance	2 → 1
dT	<code>table(\$1)</code>	Table	1 → 1

Symbol	R Code	Description	Stack Effect
dX	xtabs(\$2, \$1)	XTabs	2 → 1
h	median(\$1)	Median	1 → 1
lA	any(\$1)	Any	1 → 1
lZ	sum(\$1 != 0)	NNZ	1 → 1
m	mean(\$1)	Mean	1 → 1
mQ	quantile(\$1, \$2)	Quantile	2 → 1
mn	min(\$2, \$1)	Min2	2 → 1
mx	max(\$2, \$1)	Max2	2 → 1
r	runif(\$1)	Random (n)	1 → 1
rm	ratl_rms(\$1)	RMS	1 → 1
s	sum(\$1)	Sum	1 → 1
sc	scale(\$1)	Scale	1 → 1
sd	sd(\$1)	Standard Deviation	1 → 1
sm	summary(\$1)	Summary	1 → 1
v	var(\$1)	Variance	1 → 1
vA	all(\$1)	All	1 → 1
vd	density(\$1)	Density	1 → 1

7.8 Statistical Modeling

Symbol	R Code	Description	Stack Effect
K!	coef(\$1)	Coef	1 → 1
KA	aov(\$1)	AOV	1 → 1
KB	BIC(\$1)	BIC	1 → 1
KE	residuals(\$1)	Residuals	1 → 1
KF	fitted(\$1)	Fitted	1 → 1
KL	logLik(\$1)	LogLik	1 → 1
KV	vcov(\$1)	VCov	1 → 1
av	anova(\$1)	Anova	1 → 1
dA	aggregate(\$3, \$2, \$1)	Aggregate	3 → 1
dE	cut(\$2, \$1)	Cut	2 → 1
dY	by(\$3, \$2, \$1)	By	3 → 1
ka	anova(\$1)	Anova	1 → 1
ke	predict(\$2, \$1)	Predict	2 → 1
kg	glm(\$1)	GLM	1 → 1
ki	AIC(\$1)	AIC	1 → 1
kl	lm(\$1)	Linear Model	1 → 1
kn	nls(\$1)	NLS	1 → 1
ko	loess(\$1)	Loess	1 → 1

Symbol	R Code	Description	Stack Effect
lm	lm(\$1)	Linear Model	1 → 1
mC	confint(\$1)	Conf Int	1 → 1
mL	lm(\$2 ~ \$1)	LM Simple	2 → 1
mM	model.matrix(\$1)	Model Matrix	1 → 1
mO	offset(\$1)	Offset	1 → 1
mP	predict(\$1)	Predict	1 → 1
mR	model.frame(\$1)	Model Frame	1 → 1
mT	terms(\$1)	Terms	1 → 1
mU	update(\$2, \$1)	Update	2 → 1
mW	formula(\$1)	Formula	1 → 1
vp	prcomp(\$1)	PCA	1 → 1

7.9 Distributions & Tests

Symbol	R Code	Description	Stack Effect
DN	dnorm(\$3, \$2, \$1)	Density Normal	3 → 1
KC	cor.test(\$2, \$1)	Cor Test	2 → 1
KK	kruskal.test(\$1)	Kruskal-Wallis	1 → 1
N	rnorm(\$1)	Random Normal	1 → 1
PN	pnorm(\$3, \$2, \$1)	Prob Normal	3 → 1
QN	qnorm(\$3, \$2, \$1)	Quantile Normal	3 → 1
RB	rbeta(\$3, \$2, \$1)	Random Beta	3 → 1
RE	rexp(\$2, \$1)	Random Exponential	2 → 1
RN	rnorm(\$3, \$2, \$1)	Random Normal	3 → 1
RU	runif(\$3, \$2, \$1)	Random Uniform	3 → 1
dB	dbinom(\$3, \$2, \$1)	Density Binomial	3 → 1
dC	dchisq(\$2, \$1)	Density Chi-Square	2 → 1
dG	dgeom(\$2, \$1)	Density Geometric	2 → 1
dL	dlogis(\$3, \$2, \$1)	Density Logistic	3 → 1
dN	dnbinom(\$3, \$2, \$1)	Density Neg-Binomial	3 → 1
dP	dpois(\$2, \$1)	Density Poisson	2 → 1
db	dbeta(\$3, \$2, \$1)	Density Beta	3 → 1
dc	dcauchy(\$3, \$2, \$1)	Density Cauchy	3 → 1
de	dexp(\$2, \$1)	Density Exponential	2 → 1
df	df(\$3, \$2, \$1)	Density F	3 → 1
dg	dgamma(\$3, \$2, \$1)	Density Gamma	3 → 1
dh	dhyper(\$4, \$3, \$2, \$1)	Density Hypergeometric	4 → 1
dl	dlnorm(\$3, \$2, \$1)	Density Log-Normal	3 → 1
dn	dnorm(\$1)	Density Normal	1 → 1

Symbol	R Code	Description	Stack Effect
dt	dt(\$2, \$1)	Density Student-t	2 → 1
du	dunif(\$3, \$2, \$1)	Density Uniform	3 → 1
dw	dweibull(\$3, \$2, \$1)	Density Weibull	3 → 1
kb	bartlett.test(\$1)	Bartlett Test	1 → 1
kc	chisq.test(\$1)	Chi-Square Test	1 → 1
kf	fisher.test(\$1)	Fisher Test	1 → 1
kk	ks.test(\$2, \$1)	KS Test	2 → 1
kp	prop.test(\$2, \$1)	Prop Test	2 → 1
ks	shapiro.test(\$1)	Shapiro-Wilk	1 → 1
kt	t.test(\$2, \$1)	T-Test	2 → 1
kv	var.test(\$2, \$1)	F-Test Var	2 → 1
kw	wilcox.test(\$2, \$1)	Wilcoxon Test	2 → 1
pB	pbinom(\$3, \$2, \$1)	Prob Binomial	3 → 1
pC	pchisq(\$2, \$1)	Prob Chi-Square	2 → 1
pG	pgeom(\$2, \$1)	Prob Geometric	2 → 1
pL	plogis(\$3, \$2, \$1)	Prob Logistic	3 → 1
pN	pnbinom(\$3, \$2, \$1)	Prob Neg-Binomial	3 → 1
pP	ppois(\$2, \$1)	Prob Poisson	2 → 1
pb	pbeta(\$3, \$2, \$1)	Prob Beta	3 → 1
pc	pcauchy(\$3, \$2, \$1)	Prob Cauchy	3 → 1
pe	pexp(\$2, \$1)	Prob Exponential	2 → 1
pf	pf(\$3, \$2, \$1)	Prob F	3 → 1
pg	pgamma(\$3, \$2, \$1)	Prob Gamma	3 → 1
ph	phyper(\$4, \$3, \$2, \$1)	Prob Hypergeometric	4 → 1
pl	plnorm(\$3, \$2, \$1)	Prob Log-Normal	3 → 1
pn	pnorm(\$1)	Prob Normal	1 → 1
pt	pt(\$2, \$1)	Prob Student-t	2 → 1
pu	punif(\$3, \$2, \$1)	Prob Uniform	3 → 1
pw	pweibull(\$3, \$2, \$1)	Prob Weibull	3 → 1
qB	qbinom(\$3, \$2, \$1)	Quantile Binomial	3 → 1
qC	qchisq(\$2, \$1)	Quantile Chi-Square	2 → 1
qG	qgeom(\$2, \$1)	Quantile Geometric	2 → 1
qL	qlogis(\$3, \$2, \$1)	Quantile Logistic	3 → 1
qN	qnbinom(\$3, \$2, \$1)	Quantile Neg-Binomial	3 → 1
qP	qpois(\$2, \$1)	Quantile Poisson	2 → 1
qb	qbeta(\$3, \$2, \$1)	Quantile Beta	3 → 1
qc	qcauchy(\$3, \$2, \$1)	Quantile Cauchy	3 → 1
qe	qexp(\$2, \$1)	Quantile Exponential	2 → 1
qf	qf(\$3, \$2, \$1)	Quantile F	3 → 1
qg	qgamma(\$3, \$2, \$1)	Quantile Gamma	3 → 1
qh	qhyper(\$4, \$3, \$2, \$1)	Quantile Hypergeometric	4 → 1

Symbol	R Code	Description	Stack Effect
ql	qlnorm(\$3, \$2, \$1)	Quantile Log-Normal	3 → 1
qn	qnorm(\$1)	Quantile Normal	1 → 1
qt	qt(\$2, \$1)	Quantile Student-t	2 → 1
qu	qunif(\$3, \$2, \$1)	Quantile Uniform	3 → 1
qw	qweibull(\$3, \$2, \$1)	Quantile Weibull	3 → 1
rB	rbinom(\$3, \$2, \$1)	Random Binomial	3 → 1
rC	rchisq(\$2, \$1)	Random Chi-Square	2 → 1
rG	rgeom(\$2, \$1)	Random Geometric	2 → 1
rL	rlogis(\$3, \$2, \$1)	Random Logistic	3 → 1
rN	rnbinom(\$3, \$2, \$1)	Random Neg-Binomial	3 → 1
rP	rpois(\$2, \$1)	Random Poisson	2 → 1
rb	rbinom(\$1, size=1, prob=0.5)	Random Binomial	1 → 1
rc	rcauchy(\$3, \$2, \$1)	Random Cauchy	3 → 1
re	rexp(\$1)	Random Exp	1 → 1
rf	rf(\$3, \$2, \$1)	Random F	3 → 1
rg	rgamma(\$3, \$2, \$1)	Random Gamma	3 → 1
rh	rhyper(\$4, \$3, \$2, \$1)	Random Hypergeometric	4 → 1
rl	rlnorm(\$3, \$2, \$1)	Random Log-Normal	3 → 1
rp	rpois(\$1, lambda=1)	Random Poisson	1 → 1
rt	rt(\$2, \$1)	Random Student-t	2 → 1
rw	rweibull(\$3, \$2, \$1)	Random Weibull	3 → 1
tt	t.test(\$2, \$1)	T-Test	2 → 1

7.10 Math Functions

Symbol	R Code	Description	Stack Effect
SP	sinpi(\$1)	Sin(pi*x)	1 → 1
a2	atan2(\$2, \$1)	ArcTan2	2 → 1
aC	acos(\$1)	ArcCos	1 → 1
aS	asin(\$1)	ArcSin	1 → 1
aT	atan(\$1)	ArcTan	1 → 1
ab	abs(\$1)	Abs	1 → 1
ac	acosh(\$1)	ArcCosh	1 → 1
as	asinh(\$1)	ArcSinh	1 → 1
at	atanh(\$1)	ArcTanh	1 → 1
ch	cosh(\$1)	Cosh	1 → 1
cl	ceiling(\$1)	Ceiling	1 → 1
cp	cospi(\$1)	Cos(pi*x)	1 → 1
e-	expm1(\$1)	Exp(x)-1	1 → 1

Symbol	R Code	Description	Stack Effect
ex	exp(\$1)	Exp	1 → 1
fl	floor(\$1)	Floor	1 → 1
l+	log1p(\$1)	Log(1+x)	1 → 1
l1	log10(\$1)	Log 10	1 → 1
l2	log2(\$1)	Log 2	1 → 1
lg	log(\$1)	Log Natural	1 → 1
mc	cos(\$1)	Cos	1 → 1
ms	sin(\$1)	Sin	1 → 1
mt	tan(\$1)	Tan	1 → 1
ro	round(\$1)	Round	1 → 1
sg	sign(\$1)	Sign	1 → 1
sh	sinh(\$1)	Sinh	1 → 1
sq	sqrt(\$1)	Sqrt	1 → 1
th	tanh(\$1)	Tanh	1 → 1
tp	tanpi(\$1)	Tan(pi*x)	1 → 1
tr	trunc(\$1)	Trunc	1 → 1

7.11 Combinatorics & Special

Symbol	R Code	Description	Stack Effect
LG	lgamma(\$1)	Log Gamma	1 → 1
MC	choose(\$2, \$1)	Choose	2 → 1
XP	ratl_prime_factors(\$1)	Prime Factors	1 → 1
Xn	choose(\$2, \$1)	Choose (nCr)	2 → 1
Xq	ratl_is_prime(\$1)	Is Prime	1 → 1
di	digamma(\$1)	Digamma	1 → 1
fa	ratl_factors(\$1)	Factors	1 → 1
fp	factorial(\$1)	Factorial	1 → 1
g	ratl_gcd(\$2, \$1)	GCD	2 → 1
lb	lbeta(\$2, \$1)	Log Beta	2 → 1
lc	ratl_lcm(\$2, \$1)	LCM	2 → 1
mB	beta(\$2, \$1)	Beta	2 → 1
mG	gamma(\$1)	Gamma	1 → 1
mf	ratl_prime_factors(\$1)	Prime Factors	1 → 1
ml	ratl_lcm(\$2, \$1)	LCM	2 → 1
mp	ratl_is_prime(\$1)	Is Prime	1 → 1
ps	psigamma(\$2, \$1)	Psigamma	2 → 1
tg	trigamma(\$1)	Trigamma	1 → 1

7.12 Complex Numbers

Symbol	R Code	Description	Stack Effect
cA	Arg(\$1)	Arg	1 → 1
cI	Im(\$1)	Imag Part	1 → 1
cJ	Conj(\$1)	Conjugate	1 → 1
cM	Mod(\$1)	Modulus	1 → 1
cR	Re(\$1)	Real Part	1 → 1

7.13 String Operations

Symbol	R Code	Description	Stack Effect
C	paste0(\$2, \$1)	Concat	2 → 1
J	paste(\$2, collapse=\$1)	Join List	2 → 1
S	strsplit(\$2, \$1)[[1]]	Split String	2 → 1
S!	chartr(\$2, \$1, \$3)	Translate	3 → 1
SG	gsub(\$2, \$1, \$3)	GSub	3 → 1
Xc	chartr(\$1, \$2, \$3)	Char Translate	3 → 1
Xt	tolower(\$1)	ToLower	1 → 1
j	paste(\$2, \$1)	Join	2 → 1
rv	paste(rev(strsplit(\$1, \$2)), collapse=\$3)	Reverse String	1 → 1
sB	grepl(\$2, \$1)	Grepl	2 → 1
sG	grep(\$2, \$1)	Grep	2 → 1
sL	tolower(\$1)	ToLower	1 → 1
sR	regexpr(\$2, \$1)	Regexpr	2 → 1
sU	toupper(\$1)	ToUpper	1 → 1
sX	gregexpr(\$2, \$1)	Gregexpr	2 → 1
sf	sprintf(\$2, \$1)	Sprintf	2 → 1
sn	nchar(\$1)	NChar	1 → 1
sr	sub(\$2, \$1, \$3)	Sub	3 → 1

Symbol	R Code	Description	Stack Effect
ss	substr(\$3, \$2, \$1)	Substr	3 → 1
st	trimws(\$1)	TrimWS	1 → 1

7.14 Set Operations

Symbol	R Code	Description	Stack Effect
sD	setdiff(\$2, \$1)	SetDiff	2 → 1
sI	intersect(\$2, \$1)	Intersect	2 → 1
sN	union(\$2, \$1)	Union	2 → 1

7.15 Bitwise Operations

Symbol	R Code	Description	Stack Effect
bA	bitwAnd(\$2, \$1)	BitAnd	2 → 1
bL	bitwShiftL(\$2, \$1)	BitShiftL	2 → 1
bN	bitwNot(\$1)	BitNot	1 → 1
bO	bitwOr(\$2, \$1)	BitOr	2 → 1
bR	bitwShiftR(\$2, \$1)	BitShiftR	2 → 1
bX	bitwXor(\$2, \$1)	BitXor	2 → 1

7.16 Type & Introspection

Symbol	R Code	Description	Stack Effect
A	utf8ToInt(\$1)	To ASCII	1 → 1
AS	as(\$2, \$1)	As Type	2 → 1
Xb	ratl_bin(\$1)	To Binary	1 → 1
Xd	as.Date(\$1)	To Date	1 → 1
a	as.character(\$1)	To String	1 → 1
c	intToUtf8(\$1)	To Char	1 → 1
is	is(\$2, \$1)	Is Type	2 → 1
mF	factor(\$1)	Factor	1 → 1
n	as.numeric(\$1)	To Number	1 → 1
oA	attr(\$2, \$1)	Attr	2 → 1
oC	colnames(\$1)	Colnames	1 → 1

Symbol	R Code	Description	Stack Effect
oa	<code>attributes(\$1)</code>	Attributes	1 → 1
oc	<code>class(\$1)</code>	Class	1 → 1
od	<code>dim(\$1)</code>	Dim	1 → 1
on	<code>names(\$1)</code>	Names	1 → 1
or	<code>rownames(\$1)</code>	Rownames	1 → 1
ot	<code>typeof(\$1)</code>	Typeof	1 → 1
ou	<code>unlist(\$1)</code>	Unlist	1 → 1
zd	<code>as.Date(\$1)</code>	To Date	1 → 1

7.17 File I/O

Symbol	R Code	Description	Stack Effect
F	<code>readLines(\$1)</code>	Read File	1 → 1
FL	<code>list.files(\$1)</code>	List Files	1 → 1
FW	<code>{writeLines(as.character(\$2), \$1); ...</code>	Write File	2 → 0
W	<code>writeLines(as.character(\$2), \$1); ...</code>	Write File	2 → 0
fA	<code>dirname(\$1)</code>	Dirname	1 → 1
fC	<code>write.csv(\$2, \$1)</code>	Write CSV	2 → 0
fD	<code>list.dirs(\$1)</code>	List Dirs	1 → 1
fN	<code>file.create(\$1)</code>	File Create	1 → 1
fP	<code>normalizePath(\$1)</code>	Abs Path	1 → 1
fT	<code>tempfile()</code>	Temp File	0 → 1
fX	<code>dir.exists(\$1)</code>	Dir Exists	1 → 1
fb	<code>basename(\$1)</code>	Basename	1 → 1
fc	<code>read.csv(\$1)</code>	Read CSV	1 → 1
fd	<code>dir.create(\$1)</code>	Dir Create	1 → 1
fe	<code>file.exists(\$1)</code>	File Exists	1 → 1
fi	<code>file.info(\$1)</code>	File Info	1 → 1
fm	<code>file.remove(\$1)</code>	File Remove	1 → 1
fr	<code>read.table(\$1)</code>	Read Table	1 → 1
ft	<code>tempdir()</code>	Temp Dir	0 → 1
fw	<code>write.table(\$2, \$1)</code>	Write Table	2 → 0

7.18 System

Symbol	R Code	Description	Stack Effect
T	<code>as.numeric(Sys.time())</code>	Time Epoch	0 → 1
ZT	<code>Sys.timezone()</code>	Timezone	0 → 1
ge	<code>Sys.getenv(\$1)</code>	Getenv	1 → 1
pi	<code>Sys.getpid()</code>	GetPID	0 → 1
sE	<code>date()</code>	Date	0 → 1
sF	<code>options(prompt=\$1)</code>	SetPrompt	1 → 0
sH	<code>getwd()</code>	GetWD	0 → 1
sJ	<code>setwd(\$1)</code>	SetWD	1 → 0
sK	<code>proc.time() - \$1</code>	Toc	1 → 1
sP	<code>Sys.sleep(\$1)</code>	Sleep	1 → 0
sT	<code>proc.time()</code>	Tic	0 → 1
sV	<code>R.version.string</code>	RVersion	0 → 1
sY	<code>options(scipen=\$1)</code>	SetScipen	1 → 0
se	<code>Sys.setenv(...)</code>	Setenv	1 → 0
um	<code>Sys.umask(\$1)</code>	Umask	1 → 1
zg	<code>gc()</code>	GC	0 → 1
zl	<code>Sys.localeconv()</code>	Locale	0 → 1
zo	<code>options()</code>	Options	0 → 1
zt	<code>Sys.time()</code>	SysTime	0 → 1
zv	<code>version</code>	Version	0 → 1